# Užsakymų pristatymų maršrutų optimizavimas

Kursinio darbo I dalis: projektavimo dokumentas

## 1. Problema ir idėja

**Sistema vienu sakiniu:** Sistema skirta apskaičiuoti optimaliausią užsakymo pristatymų maršrutus.

**Problema ir dabartinis procesas:** Kurjeriai naudoja „Google Maps“ ar kitus žemėlapius suplanuoti pristatymo maršrutus. Todėl dažnai nėra optimaliausiai naudojamos kuro ir laiko sąnaudos.

**Nauda:** Kurjeriai lengviau ir greičiau pristatys užsakymus.

**Naudotojai:** Maršrutų prižiūrėtojai kurie generuos maršrutus ir kurjeriai kurie pristatinės pagal maršrutus.

**Prielaidos:** Eismo sąlygos nėra vertinamos skaičiuojant atstumą remiamasi vidutiniu greičiu.

## 2. Apimtis

| Funkcija | Ką naudotojas galės atlikti | Pagrindinis modulis ar pagalbinė funkcija |
|---|---|---|
| Maršruto generavimas | Paspaudus mygtuką automatiškai gauti pristatymo eiliškumą. | Pagrindinis |
| Maršruto koregavimas | Koreguoti sugeneruotą maršrutą | Pagalbinis |
| Maršrutų peržiūra | Matyti sugeneruotus maršrutus sąraše | Pagalbinis |

**Į kursinio darbo apimtį neįeina:** Realaus laiko GPS sekimas, mobilioji programėlė kurjeriams, adresų vertimas į koordinates

## 3. Pagrindinis modulis

**Pavadinimas ir atsakomybė:** Optimalaus maršruto generavimas. Sugeneruoti optimaliausią pristatymo maršrutą pagal pristatymo adresus.

**Logika, kurią reikės projektuoti ir testuoti:** Kelionės laiko taškų skaičiavimas pridedant iškrovimo trukmę.

**Įvestis:**

Nepriskirtų užsakymų sąrašas: Taškas A, Taškas B, Taškas C.

Fiksuota iškrovimo trukmė kiekviename taške: 5 min.

Maža atstumų matrica (kelionės trukmė minutėmis):

- Sandėlis (S) -> A: 10 min , S -> B: 15 min , S -> C: 20 min
- A -> B: 5 min , A -> C: 25 min
- B -> C: 10 min

Pradinis (neoptimizuotas) maršrutas**:** Sandėlis -> C -> A -> B -> Sandėlis.

- *Laiko skaičiavimas:* S->C (20) + iškrovimas (5) + C->A (25) + iškrovimas (5) + A->B (5) + iškrovimas (5) + B->S (15).
- *Pradinis laikas:* 80 min**.**

**Išvestis:**

Tiksli galutinė seka ir laikas:

- Seka: Sandėlis -> A -> B -> C -> Sandėlis.
- *Laiko skaičiavimas:* S->A (10) + iškrovimas (5) + A->B (5) + iškrovimas (5) + B->C (10) + iškrovimas (5) + C->S (20).
- *Galutinis laikas:* 60 min**.**

**Veikimo eiga:** Nuskaitomi nepriskirti užsakymai su atstumų matrica, skaičiuojami atstumai ir apytiksliai laikai tarp užsakymų, sugeneruojamas maršrutas.

### Taisyklės arba sprendimo žingsniai

1. Maršrutas turi prasidėti sandėlyje ir pasibaigti sandėlyje.
2. Galutinio maršruto atstumas ar laikas turi būti mažesnis negu pradinio neoptimizuotas maršruto.
3. Kiekvienas pristatymo adresas aplankomas tik vieną kartą.

### Scenarijai būsimiems testams

| Scenarijus | Pradinės sąlygos ir konkreti įvestis | Veiksmas | Tikslus laukiamas rezultatas |
|---|---|---|---|
| Įprastas atvejis | Pateikiamas nepriskirtų užsakymų sąrašas ir atstumų matrica. Pvz: Užsakymai A, B, C ir 3 skyriaus atstumų matrica (iškrovimas 5 min). | Iškviečiamas maršruto optimizavimo modulis. | Grąžinamas maršrutas S -> A -> B -> C -> S, kurio bendra trukmė yra tiksli. |
| Ribinis atvejis arba konfliktas | Pateikiamame užsakymų sąraše yra du vienodi adresai su skirtingais užsakymais | Iškviečiamas maršruto optimizavimo modulis | Gražinamas maršrutas kuriame pasikartojantys adresai apdorojame kaip esantys greta vienas kito |
| Klaida arba neįmanomas rezultatas | Užsakymas neturi pristatymo adreso ar adresas nežinomas | Iškviečiamas optimizavimo modulis | Maršrutas negeneruojamas išmetama klaida. |

| Scenarijus | Pradinės sąlygos ir konkreti įvestis | Veiksmas | Tikslus laukiamas rezultatas |
|---|---|---|---|
| Pradinis maršrutas jau optimalus | Įvedamas užsakymų sąrašas, kurio pradinė seka jau atitinka trumpiausią įmanomą laiką | Iškviečiamas maršruto optimizavimo modulis. | Modulis grąžina tiksliai tokią pačią seką kaip ir pradinė. Bendra maršruto trukmė nesikeičia, algoritmas patvirtina, kad optimizuoti labiau neįmanoma. |

**Jei modulis naudoja AI:** Netaikoma

## 4. Kokybės atributas

**Pasirinktas atributas:** Našumas

**Kodėl svarbus šiai sistemai:** Maršruto skaičiavimo paieška yra matematiškai sunkus uždavinys kuris gali trukti ilgai, optimizuoto maršruto reikia greitai per kelias sekundes.

**Tikrinimo scenarijus ir sąlygos:** Į sistemą paduodamas maksimalus leistinas 500 adresų skaičius, maršruto generavimas paleidžiamas vietinėje kompiuterio aplinkoje. Optimizavimo kriterijus: bendra maršruto trukmė (kelionės laikas + iškrovimas).

**Sėkmės kriterijus:** Modulis maršrutą visiems 500 adresų apskaičiuoja ne ilgiau kaip per 5 sekundes.

**Numatytas projektavimo sprendimas:** Maršrutų generavimui bus naudojamas artimiausio kaimyno algoritmas.

**Kaip patikrinsiu vėlesniame etape:** Paduodama atsitiktinių adresų matrica kuri bus paleidžiama per maršruto generavimo moduli ir apskaičiuojama modulio apskaičiavimo greitis.

**Sprendimo kaina arba ribojimas:** Atsižvelgiant į kompiuterinę galią nebus užtikrintas tobuliausias maršrutas.

## 5. Pradinė sistemos struktūra

### Paprasta schema

Vartotojo įvestis -> Optimizavimo modulis -> Laikini duomenys ->Terminalo ar lango išvestis

| Sistemos dalis | Atsakomybė |
|---|---|
| Vartotojo įvestis / vartotojo sąsaja | Nuskaityti užsakymų adresus, parodyti gautą maršrutą |
| Maršruto generavimo modulis | Apskaičiuoti optimaliausia maršrutą |

**Planuojamos technologijos ir pasirinkimo priežastys:** Bus naudojama Python programavimo kalba, vartotojo sąsajai galbūt naudojama „Streamlit“ sistema.

## 6. AI panaudojimas

### AI rengiant šį dokumentą

Taip, naudojau AI.

| Priemonė ir užduotis | Ką panaudojau | Ką atmečiau arba perrašiau ir kodėl | Kaip patikrinau |
|---|---|---|---|
| „Gemini“ idėjos susiaurinimui | Idėja | Susiaurinau temą iki labiau paprasčiau | Pagal asmeninius įgūdžius |

### Planuojamas AI naudojimas kuriant sistemą

**Kur ir kam naudosiu AI:** Sistemos kodo optimizavimui ir dalis kodo generavimas.

**Kaip tikrinsiu pasiūlymus ir sugeneruotą kodą:** Pagal interneto pavyzdžius, savo supratimą.

**Ar AI bus sistemos funkcionalumo dalis:** Ne

## 7. Tolesnių darbų planas

| Darbas | Apčiuopiamas rezultatas | Planuojama darbų seka |
|---|---|---|
| Pradinių duomenų skaičiavimui parengimas | Funkcijos skaičiuojančios atstumus tarp adresų taškų | Sukurti atstumų matricos generavimą |
| Optimizavimo algoritmas | Pagrindinis maršruto rikiavimo algoritmas | Sukurti artimiausio kaimyno algoritmą |
| Vartotojo sąsaja | Veikiantis terminalo ar aplikacijos būdas parodanti maršrutą | Sukurti terminalo ar aplikacijos atvaizdavimą |

**Būsimo prototipo veikimo scenarijus:** Pagrindinio maršruto generavimo funkcija duodami užsakymai su adresais, gražinamas optimalus maršrutas. Atstumai tarp taškų bus gaunami iš iš anksto sugeneruotos atstumų matricos (CSV failo), kurioje jau bus pateiktos kelionės trukmės tarp visų galimų pristatymo taškų ir sandėlio. Sistema adresus naudos tik kaip identifikatorius (raktus) šiai matricai nuskaityti.

| Rizika arba neaiškumas | Kaip patikrinsiu arba sumažinsiu |
|---|---|
| Didelis adresų kiekis ir negalėjimas jų greitai apdoroti | Naudojamas algoritmas kuris randa pakankamai gera sprendimą greitai. |

## Šaltiniai, jei naudojote